

Objetivo

O objetivo desta atividade prática em laboratório é implementar uma **tabela hash** para armazenar cadastros de estudantes. A chave de busca será o número de matrícula, e o tratamento de colisões deverá ser feito por **encadeamento separado**, isto é, cada posição da tabela armazenará uma lista de estudantes que produziram o mesmo índice pela função hash.

Ao final, o programa deverá permitir inserir, buscar, atualizar, remover e imprimir cadastros armazenados na tabela hash.

Parte 1 – Implementação da Tabela Hash

1. Inicialmente crie os seguintes arquivos fonte:

- **Aluno**: classe que representa um estudante cadastrado na tabela;
- **TabelaHash**: classe que representa a tabela hash;
- **TabelaHashMain**: classe com o método principal para demonstrar o funcionamento da tabela hash.

2. Estrutura sugerida para a classe **Aluno**:

```
public class Aluno {
    private int matricula;
    private String nome;
    private String email;
    private double mediaGeral;
    private Aluno prox;

    public Aluno(int matricula, String nome,
                 String email, double mediaGeral) {
        this.matricula = matricula;
        this.nome = nome;
        this.email = email;
        this.mediaGeral = mediaGeral;
        this.prox = null;
    }

    // getters, setters e toString
}
```

3. Significado dos atributos:

- **matricula**: chave usada para inserir, buscar e remover estudantes;
- **nome**: nome do estudante;
- **email**: e-mail do estudante;
- **mediaGeral**: média geral do estudante;
- **prox**: referência para o próximo estudante na lista de colisão.

4. A classe `TabelaHash` deve possuir, no mínimo, os seguintes atributos:

```
private Aluno[] tabela;
private int tamanho;
```

5. Métodos a serem implementados na classe `TabelaHash`:

- `public TabelaHash(int tamanho)`: construtor que instancia o vetor da tabela e inicializa todas as posições com `null`;
- `private int hash(int matricula)`: calcula o índice da tabela a partir da matrícula;
- `public Aluno get(int matricula)`: busca e retorna o estudante com a matrícula informada, ou `null` se não for encontrado;
- `public void set(int matricula, String nome, String email, double mediaGeral)`: insere um novo estudante ou atualiza os dados de um estudante já cadastrado;
- `public void remove(int matricula)`: remove o estudante com a matrícula informada, se ele existir;
- `public boolean contem(int matricula)`: retorna `true` se a matrícula estiver cadastrada, ou `false` caso contrário;
- `public int quantidade()`: retorna a quantidade total de estudantes cadastrados;
- `public int colisoes()`: retorna a quantidade de elementos que estão em posições com colisão;
- `public int maiorLista()`: retorna o tamanho da maior lista encadeada da tabela;
- `public String toString()`: retorna uma representação textual do conteúdo completo da tabela hash.

6. A função hash deve usar o método da divisão:

```
indice = matricula % tamanho;
```

Use um valor primo para o tamanho da tabela. Nesta atividade, use inicialmente 127.

7. O método `set` deve primeiro verificar se a matrícula já existe na tabela:

- se a matrícula já existir, atualize os campos **nome**, **email** e **mediaGeral**;
- se a matrícula não existir, crie um novo objeto `Aluno` e insira-o na lista correspondente ao índice calculado pela função hash.

8. A remoção deve funcionar corretamente em três situações:

- remoção do primeiro elemento da lista;
- remoção de um elemento no meio ou no final da lista;
- tentativa de remoção de uma matrícula inexistente.

9. O método `colisoas` deve ajudar a observar o efeito das colisões. Uma forma simples de contabilizar é somar, em cada posição da tabela, os elementos excedentes ao primeiro. Por exemplo, uma lista com 4 elementos contribui com 3 colisões.

Parte 2 – Problema Aplicado: Cadastro de Estudantes

Após implementar a tabela hash, utilize-a para representar um pequeno cadastro de estudantes indexado pelo número de matrícula.

Implemente, na classe `TabelaHashMain`, as seguintes funcionalidades:

1. Criar uma tabela hash vazia;
2. Inserir pelo menos 15 estudantes;
3. Incluir, entre os dados de teste, matrículas que produzam colisões de propósito;
4. Buscar pelo menos três matrículas existentes;
5. Buscar pelo menos três matrículas inexistentes;
6. Atualizar os dados de um estudante já cadastrado;
7. Remover um estudante que esteja no início de uma lista de colisão;
8. Remover um estudante que esteja no meio ou no final de uma lista de colisão;
9. Tentar remover uma matrícula inexistente;
10. Imprimir a tabela hash completa;
11. Informar a quantidade total de estudantes cadastrados;
12. Informar a quantidade de colisões;
13. Informar o tamanho da maior lista da tabela.

Sugestão de matrículas para gerar colisões com tamanho 127:

1001, 1128, 1255, 1382
2002, 2129, 2256
3003, 3130
4010, 5020, 6030, 7040, 8050, 9060

As matrículas 1001, 1128, 1255 e 1382 geram o mesmo resto na divisão por 127, portanto devem cair na mesma posição da tabela.

Questões para Análise

Após executar os testes, responda no próprio código, em forma de comentários, ou em um pequeno arquivo de texto:

1. Qual é a função da operação hash em uma tabela hash?
2. O que é uma colisão?
3. Como o encadeamento separado resolve colisões?

4. Por que é recomendável usar um tamanho primo para a tabela?
5. O que acontece com o desempenho das buscas se muitas chaves caírem na mesma posição da tabela?

Observações

- A tabela deve usar encadeamento separado com listas encadeadas;
- A chave de busca deve ser a matrícula do estudante;
- Não permita duas entradas com a mesma matrícula; nesses casos, atualize o cadastro existente;
- Utilize adequadamente os princípios de encapsulamento da programação orientada a objetos;
- Organize a saída do método `toString` de forma que seja possível visualizar as colisões em cada posição da tabela;
- Implemente na classe `TabelaHashMain` um exemplo completo demonstrando o funcionamento da tabela hash.

Observação Final

Ao final da atividade, o programa deve demonstrar claramente como uma tabela hash transforma uma chave em índice, como colisões podem ocorrer e como listas encadeadas permitem armazenar múltiplos elementos associados à mesma posição da tabela.